# Yeşil Kodlama (Green Coding) ve Dijital Karbon Ayak İzi Optimizasyon Aracı

**Sakarya Üniversitesi** · Toplumsal Katkı ve Sürdürülebilirlik Koordinatörlüğü COP31 projesi
Proje ekibi: Yunus Emre Sevindik · Mehmet Zahid Görgeç · Sürüm: Ekim 2026 · Açık kaynak: github.com/yes3760/yesil-kod

ÖZET

Yeşil Kod, Python kodunda enerji israf eden kalıpları **bulur**, kodu kendisi **yeniden yazar**, iki sürümün birebir aynı sonucu verdiğini **doğrular** ve aynı iş için harcanan enerjiyi **ölçüp** Türkiye şebekesinin karbon yoğunluğuyla $gCO_2e$'ye çevirir. Dört gerçekçi senaryonun dördünde de düzeltmeyi motor üretti; işlemci enerjisi 2,4 ile 4.049 kat arasında azaldı ve çıktılar değişmedi.

**465×**
yalnızca bir satır düzeltmeyle, aynı sonuç

## 01 Problem

Veri merkezlerinin elektrik tüketiminin 2024'teki **415 TWh** düzeyinden 2030'da **~945 TWh**'a çıkması bekleniyor (IEA, 2025). Bir döngüdeki tek satırlık israf, günde milyonlarca kez çalıştığında gerçek bir elektrik ve karbon yüküne dönüşüyor; ama kod satırının enerjisi hiçbir yerde ölçülmüyor ve gösterilmiyor.

## 02 Çözüm: dört adım

### 1 Bul

Kod çalıştırılmadan sözdizimi ağacı (AST) taranır; 113 israf kalıbı satır satır işaretlenir.

### 2 Düzelt

Motor on bir kalıbı yeniden yazar; yalnızca değişen satırlara dokunur, yorumlar ve biçim korunur.

### 3 Doğrula

Özgün ve düzeltilmiş kod aynı veriyle çalıştırılır; çıktı birebir aynı değilse düzeltme reddedilir.

### 4 Ölç

İki sürüm aynı sayıda isteği işler; işlemci enerjisi ölçülür ve karbona çevrilir.

| KATEGORİ | ÖRNEK KALIPLAR | MOTORUN KENDİ DÜZELTTİKLERİ |
|---|---|---|
| **Veritabanı · 14** | N+1 sorgu, döngüde commit, tek tek INSERT, SELECT *, ORDER BY RANDOM(), Python'da filtreleme ve toplama | **N+1 → toplu IN sorgusu** |
| **Döngü ve algoritma · 18** | Listede arama, değişmeyen iş, önbelleksiz özyineleme, min/max için sıralama, ilk eşleşme için tam liste, döngüde kopya | **set(), döngü dışına taşıma, lru_cache, min()/max()** |
| **Veri yapıları ve Python dili · 31** | deque yerine liste, elle kurgu, gereksiz çift dönüşüm, any/sum içinde liste, Counter/defaultdict yerine elle sözlük | **Gereksiz kopya ve dönüşümleri kaldırma (5 kalıp)** |
| **Metin ve düzenli ifadeler · 7** | Döngüde += ile birleştirme, düz metin için regex, zincirleme replace, find() ile içerme | Öneri |
| **Veri bilimi (pandas, NumPy) · 17** | iterrows, döngüde concat/append, apply ile vektörleştirilebilir iş, np.vectorize, CSV yerine Parquet | Öneri: vektörel işlemler |
| **Makine öğrenmesi · 4** | Tek tek tahmin, döngüde .item(), çıkarımda no_grad eksikliği | Öneri: toplu tahmin, no_grad |
| **Ağ ve bulut · 7** | Oturumsuz HTTP, döngüde bağlantı/istemci, Redis ve S3'e tek tek istek, zaman aşımsız istek | Öneri: bağlantıyı yeniden kullanma |
| **Dosya ve G/Ç · 8** | Döngüde dosya/model yükleme, flush/fsync, print, readlines, dizini yeniden tarama | Öneri: bir kez yükle, toplu yaz |
| **Eşzamanlılık ve bekleme · 7** | Polling, boş döngüyle bekleme, çok sık uyanma, async içinde time.sleep, döngüde süreç | Öneri: olay tabanlı bekleme |

## 03 Nasıl kullanılır

- **Canlı panel:** stant ekranında düzeltme, eşdeğerlik testi ve enerji yarışı.
- **Komut satırı:** analiz, düzeltme, ölçüm ve rapor.
- **Sürekli entegrasyon:** her pull request'te rapor; yüksek bulguda kontrol durur.
- **Web:** ziyaretçi kendi kodunu telefonunda analiz eder ve senaryoları telefonunda ölçer; kod cihazdan çıkmaz.

## 04 Teknik yapı

- Yalnızca Python standart kütüphanesi; kurulum ve internet gerektirmez.
- Linux'ta Intel/AMD **RAPL** sayaçları, Mac'te Apple'ın işlemci güç verisi (**powermetrics**); izin yoksa işlemci süresi × çekirdek gücü.
- 33 birim testi; her değişiklikte GitHub Actions ile Python 3.9 ve 3.13'te otomatik çalışır.

## 05 Sonuçlar

Her senaryo gerçek projelerde sık görülen bir israfı temsil eder. Düzeltilmiş kodların hiçbiri elle yazılmadı; hepsini motor üretti ve hepsi eşdeğerlik testinden geçti.

| SENARYO | İSRAF → MOTORUN DÜZELTMESİ | GERÇEK HAYATTA | ENERJİ AZALMASI |
|---|---|---|---|
| **Rota sayısı** | önbelleksiz çift özyineleme → lru_cache | rota, fiyat ve öneri hesapları | **3.640–4.049×** |
| **Yoklama kontrolü** | 4.500 numara, 6.000'lik listede aranıyor → set() | yoklama, stok, yetki kontrolleri | **456–698×** |
| **Fiyat listesi** | ayar ve desen her üründe yeniden hazırlanıyor → döngü dışına | her istekte ayar okuyan API'ler | **51–55×** |
| **Sipariş raporu** | 4.000 sipariş için ayrı müşteri sorgusu → toplu IN | e-ticaret listeleri, paneller | **2,4–2,9×** |

İstek başına işlemci enerjisindeki azalma; 26 Ekim 2026'da aynı bilgisayarda (Apple M4) dört ayrı ölçüm, tahmini mod. Her ölçüm 5 tekrarın ortalamasıdır. Sipariş raporunda veritabanı sorgu sayısı 4.001'den 2'ye iner; veritabanı aynı bilgisayarda olduğundan ağ tasarrufu bu orana dahil değildir. Yoklama satırı günde 1 milyon istekte yılda yaklaşık 41–89 kg $CO_2$e salıyordu.

## 06 Ölçüm yöntemi

- **Karbon:** SCI = (E × I + M) / R. E ölçülen enerji, I Türkiye şebekesi 341 g$CO_2$e/kWh (Ağu 2025–Tem 2026), R istek başına.
- **Adalet:** kısa süren kod, sayaçların ölçebileceği süreye kadar tekrarlanır; her sonuç 5 tekrarın ortalamasıdır.
- **Önbellek:** düzeltilmiş koddaki önbellekler her istekten önce temizlenir; kazanç önceki çalıştırmadan gelmez.
- **Boşta tüketim:** donanım ölçümünde bilgisayarın boştaki gücü ölçülür ve her sonuçtan çıkarılır.

## 07 Dürüst sınırlar

- Yalnızca işlemci enerjisi ölçülür; bellek, ağ, disk ve soğutma dahil değildir. Gösterilen kazanç alt sınırdır.
- Tahmini modda mutlak değerlerden çok oranlar güvenilirdir; oranlar makineden makineye değişir.
- Düzeltme motoru şimdilik Python içindir; 113 kalıptan on birini otomatik düzeltir, diğerleri için öneri verir.
- Karbon yoğunluğu yıllık ortalamadır; şebeke saatten saate değişir.

## 08 Yol haritası

- **Daha fazla dil:** JavaScript/TypeScript ve Java için aynı dört adımlı yapı.
- **Tam SCI:** donanımın gömülü emisyonunu (M) ve bellek enerjisini hesaba katmak.
- **Karbon-farkında zamanlama:** EPİAŞ'ın saatlik üretim verisiyle ağır işleri şebekenin temiz olduğu saatlere kaydırmak.
- **Geliştirici araçları:** editör eklentisi ve hazır sürekli entegrasyon adımı.
- **Pilot:** üniversitedeki öğrenci yazılım projelerinde israf taraması ve ölçülen tasarrufun raporlanması.

## 09 Sürdürülebilir Kalkınma Amaçları

**7** Erişilebilir ve temiz enerji: aynı işi daha az elektrikle yapmak

**9** Sanayi, yenilikçilik ve altyapı: verimli dijital altyapı

**12** Sorumlu üretim ve tüketim: hesaplama kaynağını israf etmemek

**13** İklim eylemi: yazılımın karbonunu ölçülebilir kılmak

Proje sayfası

**yes3760.github.io/yesil-kod** · film, sonuçlar ve telefonda deneme
**github.com/yes3760/yesil-kod** · kaynak kod, kurulum ve testler
Kaynaklar: IEA, Energy and AI (2025) · Low-Carbon Power, Türkiye (Ağu 2025–Tem 2026) · Green Software Foundation, Software Carbon Intensity (ISO/IEC 21031:2024)

Kaynak kod